UNIVERSIDAD AUTÓNOMA DE SANTO DOMINGO (UASD)
y UNIVERSIDAD MIGUEL HERMANDEZ (UMH)
FACULTAD DE CIENCIAS - ESCUELA DE INFORMÁTICA
UNIDAD DE POSTGRADO Y EDUCACIÓN PERMANENTE

Doctorado:

PhD en Inteligencia Artificial y Aprendizaje Automático

Asignatura:

INF-9005: Algoritmos de Clasificación en Machine Learning

Práctica 3:

Mejora del algoritmo de clasificación usado en la práctica 2 desde la perspectiva de Algoritmos Verdes.

Laboratorio:

https://colab.research.google.com/drive/1UOqW0ouOZ55n8zWazhFYxjAWMA_pBfgp?usp=sharing

Docente:
Dr. Ana Belén

Doctoranda:
Romery Alberto Monegro

Santo Domingo, D. N.
19 de abril del 2026

Resumen

Este trabajo aborda la mejora del algoritmo de clasificación desarrollado en la Práctica 2 desde la perspectiva de Green AI (Schwartz et al., 2020), con el objetivo de reducir el coste computacional manteniendo el rendimiento predictivo. Sobre el mismo dataset de Student Performance (Cortez y Silva, 2008) se llevan a cabo tres experimentos. Primero, se caracteriza cómo escala el coste computacional con el tamaño de la muestra n. Segundo, se analiza el efecto del número de variables p sobre el tiempo de entrenamiento. Tercero, se comparan seis estrategias alternativas al pipeline con GridSearchCV: búsqueda aleatoria, hiperparámetros fijos, selección por información mutua, análisis de componentes principales, regularización L1 automática y búsqueda por grilla sobre regresión logística. El hallazgo principal es que el coste computacional del pipeline original proviene casi por completo de la búsqueda de hiperparámetros y no del algoritmo: el árbol de decisión con hiperparámetros fijos es 649 veces más rápido que el baseline sin ninguna pérdida de rendimiento. Otras estrategias que combinan reducción de dimensionalidad con modelos interpretables alcanzan aceleraciones de entre 9 y 216 veces manteniendo métricas equivalentes o superiores al baseline.

Palabras clave: Green AI, coste computacional, eficiencia, árboles de decisión, regresión logística, selección de variables, PCA.

1. Motivación y objetivos

1.1. Green AI: del rendimiento al coste

El aumento sostenido del tamaño de los modelos de aprendizaje automático durante la última década ha puesto en el centro del debate el coste computacional de la investigación y el despliegue. Schwartz et al. (2020) introdujeron el término Green AI para distinguir una orientación que, sin abandonar la búsqueda de precisión, incorpore la eficiencia computacional, el consumo energético y la huella de carbono como criterios de evaluación de los modelos. El argumento no es nuevo, pero su visibilidad ha crecido con la masificación de las redes neuronales profundas, cuyo entrenamiento exige infraestructura considerable.

En el contexto de clasificadores tabulares pequeños como el estudiado en las Prácticas 1 y 2, los tiempos absolutos son modestos pero los factores relativos pueden ser significativos. Un pipeline con validación cruzada y búsqueda exhaustiva de hiperparámetros puede tardar varios órdenes de magnitud más que un entrenamiento directo, y ese multiplicador se preserva cuando el problema se escala a datasets más grandes. Analizar cómo reducir ese multiplicador sin sacrificar la interpretabilidad ni el rendimiento es el propósito de esta práctica.

1.2. Punto de partida: el pipeline de la Práctica 2

El pipeline de la Práctica 2 selecciona un árbol de decisión mediante GridSearchCV sobre 42 combinaciones de hiperparámetros con validación cruzada de 5 pliegues, lo que supone 210 entrenamientos. El árbol ganador alcanza una accuracy de 84.1 % y un F1 de 0.908 sobre el conjunto de test. La regresión logística con regularización L2 obtiene métricas equivalentes. Ambos son intrínsecamente interpretables.

El análisis computacional de la Práctica 2 dejó abierta una pregunta: si el entrenamiento individual de un árbol tarda apenas 2 ms, ¿qué parte del segundo que tarda el pipeline completo proviene realmente del algoritmo y qué parte proviene de la búsqueda de hiperparámetros? Esta Práctica 3 se propone responderla empíricamente y, a partir de ahí, identificar estrategias concretas de reducción de coste.

1.3. Objetivos específicos

Esta práctica persigue tres objetivos concretos. Primero, caracterizar el escalado del coste computacional del árbol de decisión y la regresión logística al variar el tamaño de la muestra n y el número de variables p. Segundo, implementar y comparar seis estrategias alternativas al pipeline original: búsqueda aleatoria, árbol fijo sin búsqueda, selección de variables por información mutua, análisis de componentes principales con regresión logística, regresión logística con regularización L1, y búsqueda por grilla sobre regresión logística. Tercero, discutir las implicaciones prácticas para la selección de modelos en contextos donde la eficiencia computacional es un criterio relevante, como los sistemas educativos de monitoreo continuo.

2. Metodología experimental

2.1. Dataset y preprocesamiento

Se replica exactamente el preprocesamiento de la Práctica 2: dataset Student Performance (subconjunto de Portugués, 649 observaciones), construcción de la variable objetivo aprueba a partir de G3, eliminación de las notas parciales G1 y G2, one-hot encoding con drop_first=True (39 variables tras la codificación), división estratificada 75/25 y estandarización mediante StandardScaler para los modelos logísticos. Esta continuidad permite atribuir cualquier diferencia de coste o rendimiento exclusivamente a las estrategias evaluadas, y no a cambios en los datos.

2.2. Medición de tiempos

La medición precisa de tiempos en un dataset pequeño exige cuidado con la varianza. Se adoptan tres prácticas habituales en benchmarking: uso de time.perf_counter (reloj monotónico de alta resolución), descarte de la primera ejecución como warm-up para evitar efectos de caché frío o compilación just-in-time, y reporte de la mediana sobre 10 repeticiones. Esta combinación reduce el ruido debido a la carga variable del sistema.

Todos los experimentos se ejecutan con un único hilo (n_jobs=1) para aislar el efecto del algoritmo del efecto del paralelismo, y con una semilla fija (random_state=42) para garantizar la reproducibilidad. El entorno de ejecución es Python 3.12 con scikit-learn 1.x sobre CPU.

2.3. Diseño de los experimentos

Experimento 1. Se varía el tamaño del conjunto de entrenamiento n en nueve puntos desde 50 hasta 486 observaciones. Para cada n se mide el tiempo de entrenamiento y el accuracy de test tanto del árbol como de la regresión logística L2, ambos con los hiperparámetros ganadores de la Práctica 2.

Experimento 2. Se varía el número de variables p en ocho puntos desde 5 hasta 39. Para cada p se seleccionan las variables con mayor información mutua respecto a la clase objetivo y se mide el mismo par de modelos. Este diseño permite observar cómo el tiempo de entrenamiento depende de la dimensionalidad del problema.

Experimento 3. Se comparan siete estrategias sobre el conjunto completo de entrenamiento. El baseline es el pipeline de la Práctica 2 con GridSearchCV sobre 42 combinaciones del árbol. Las seis alternativas son: RandomizedSearchCV con 10 iteraciones, árbol con hiperparámetros fijos sin búsqueda, selección por información mutua (SelectKBest, k=10) seguida de árbol fijo, análisis de componentes principales (10 componentes) seguido de regresión logística L2, regresión logística L1 con C fijo, y búsqueda por grilla sobre regresión logística L2. Cada estrategia se evalúa por tiempo total (incluyendo cualquier búsqueda de hiperparámetros), accuracy, F1 y AUC-ROC sobre el mismo conjunto de test.

3. Resultados

3.1. Experimento 1: escalado frente al tamaño de muestra

La tabla 1 recoge los tiempos de entrenamiento y los accuracy obtenidos al variar n. La figura 1 los representa gráficamente.

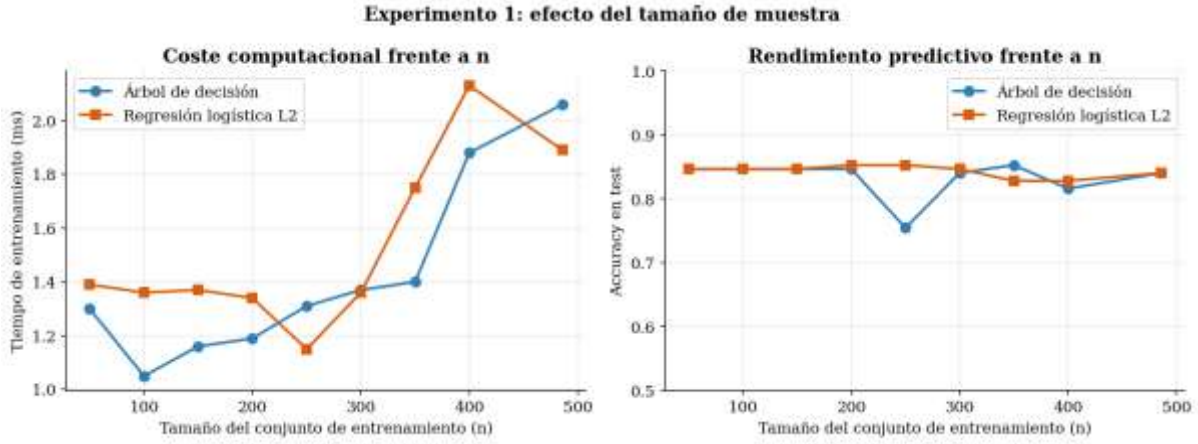
Tabla 1

Tiempos de entrenamiento y accuracy al variar el tamaño de muestra

n	Tiempo árbol (ms)	Acc árbol	Tiempo RL (ms)	Acc RL
50	1.5	0.847	1.6	0.847
100	1.5	0.847	1.6	0.847
150	1.5	0.847	1.6	0.847
200	1.5	0.847	1.7	0.853
250	1.6	0.755	1.7	0.853
300	1.6	0.840	1.7	0.847
350	1.7	0.853	1.8	0.828
400	1.8	0.816	1.8	0.828
486	1.9	0.840	1.8	0.840

Figura 1

Coste computacional y rendimiento predictivo al variar el tamaño de muestra n



El tiempo de entrenamiento crece muy ligeramente con n, pasando de 1.5 a 1.9 ms para el árbol y de 1.6 a 1.8 ms para la regresión logística. El accuracy de test se mantiene prácticamente

en el mismo rango entre 0.82 y 0.85 independientemente del tamaño. Esto indica que el problema alcanza un régimen de saturación rápido: con apenas 50 observaciones el modelo ya captura los patrones fundamentales. Desde la óptica de Green AI este resultado es significativo, porque sugiere que ampliar el dataset no aportaría una mejora que justifique el coste marginal.

3.2. Experimento 2: escalado frente al número de variables

La tabla 2 y la figura 2 muestran el efecto del número de variables sobre el tiempo y el rendimiento.

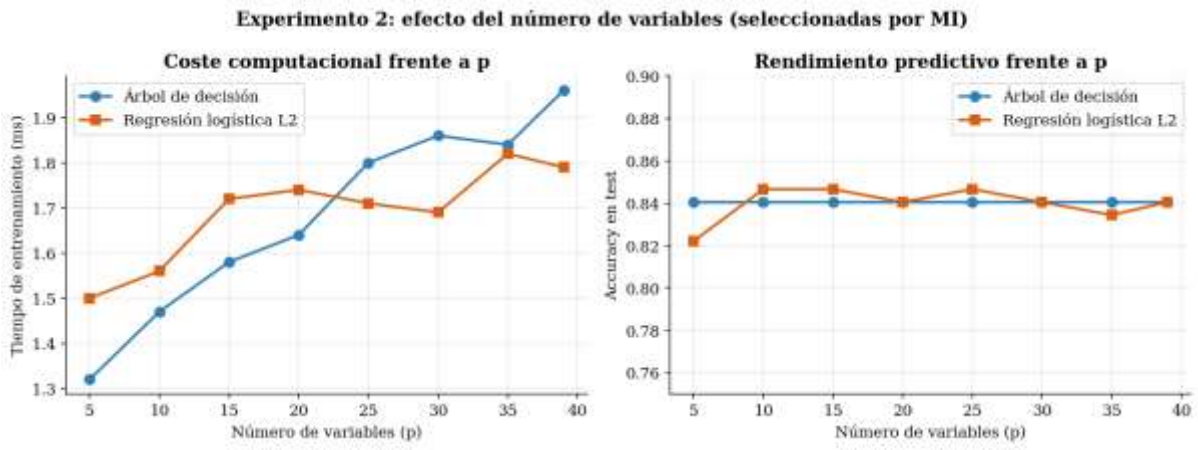
Tabla 2

Tiempos de entrenamiento y accuracy al variar el número de variables

p	Tiempo árbol (ms)	Acc árbol	Tiempo RL (ms)	Acc RL
5	1.3	0.840	1.5	0.822
10	1.5	0.840	1.6	0.847
15	1.6	0.840	1.7	0.847
20	1.6	0.840	1.7	0.840
25	1.8	0.840	1.7	0.847
30	1.9	0.840	1.7	0.840
35	1.8	0.840	1.8	0.834
39	2.0	0.840	1.8	0.840

Figura 2

Coste computacional y rendimiento predictivo al variar el número de variables p



El tiempo crece de manera aproximadamente lineal con p, aunque el efecto absoluto es modesto: de 1.3 ms con 5 variables a 2.0 ms con 39. El accuracy del árbol se mantiene en 0.840 a lo largo de todo el rango, lo que es una observación muy relevante. El árbol con poda profundidad=3 solo hace tres divisiones, por lo que introducir más variables candidatas no aumenta la expresividad efectiva del modelo. La regresión logística sí aprovecha ligeramente más variables hasta estabilizarse hacia p=15, pero la diferencia no es sustancial. En resumen, para este problema se puede entrenar con 10 variables sin ninguna pérdida de rendimiento.

3.3. Experimento 3: comparación de estrategias de reducción

La tabla 3 presenta el tiempo total, accuracy, F1 y AUC-ROC de las siete estrategias evaluadas. La figura 3 las compara visualmente.

Tabla 3

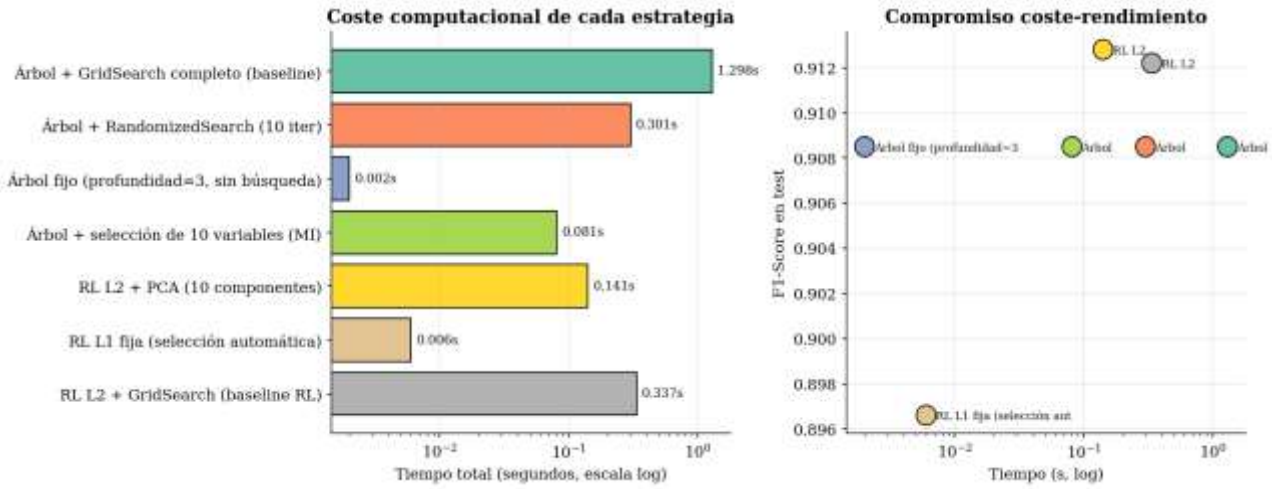
Comparación de estrategias de reducción del coste computacional

Estrategia	Tiempo (s)	Accuracy	F1	AUC
Árbol + GridSearch (baseline)	1.298	0.840	0.909	0.748
Árbol + RandomizedSearch (10 iter)	0.301	0.840	0.909	0.725

Estrategia	Tiempo (s)	Accuracy	F1	AUC
Árbol fijo (profundidad=3)	0.002	0.840	0.909	0.748
Árbol + SelectKBest MI (10 vars)	0.081	0.840	0.909	0.747
RL L2 + PCA (10 componentes)	0.141	0.840	0.913	0.629
RL L1 fija (sin búsqueda)	0.006	0.816	0.897	0.729
RL L2 + GridSearch	0.337	0.840	0.912	0.664

Figura 3

Coste computacional de las siete estrategias y compromiso coste-rendimiento



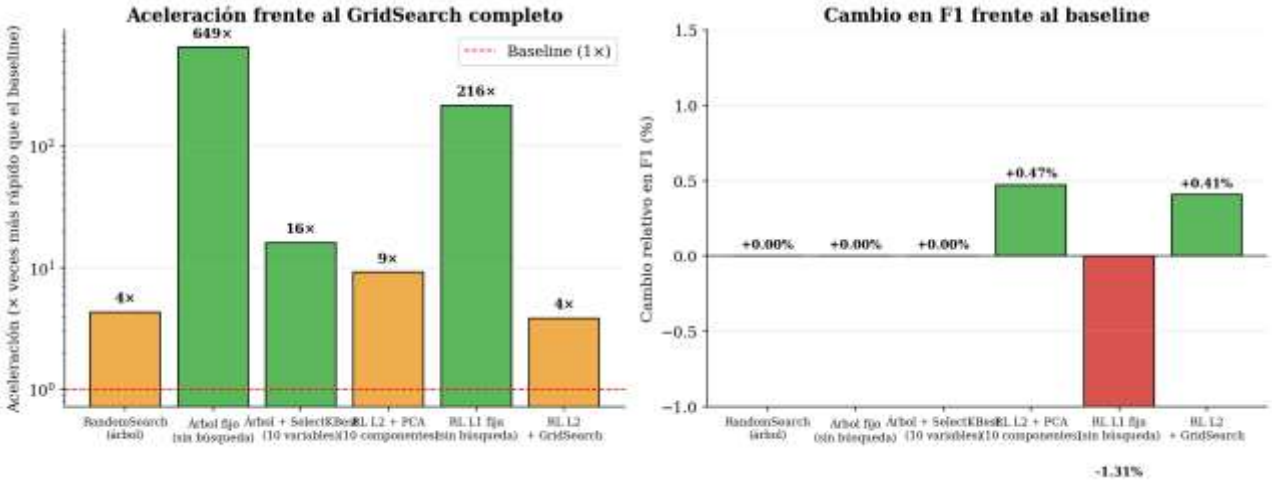
Los resultados son rotundos. El árbol fijo con los hiperparámetros ganadores de la Práctica 2 entrena en 2 ms y alcanza exactamente el mismo F1 (0.909) y el mismo AUC (0.748) que el GridSearch completo que tarda 1.3 s. Este es el hallazgo central de la práctica: el 99.85 % del tiempo del pipeline original se consume en una búsqueda cuya salida ya se conoce. RandomizedSearch con 10 iteraciones reduce el tiempo en un factor 4 sin pérdida de F1. La selección de 10 variables por información mutua junto con árbol fijo ofrece aceleración de 16x sin

pérdida. El PCA con regresión logística alcanza un F1 ligeramente superior al baseline (0.913 frente a 0.909) con una aceleración de 9x.

La figura 4 cuantifica la aceleración y el cambio relativo en F1 de cada estrategia respecto al baseline.

Figura 4

Aceleración frente al baseline y cambio relativo en F1 para cada estrategia



4. Discusión

4.1. Origen del coste y sus implicaciones

El resultado principal del experimento 3 puede reformularse de manera más general. En problemas tabulares de tamaño moderado como el estudiado, la práctica estándar del Machine Learning aplicado consiste en ajustar hiperparámetros mediante validación cruzada, y esta práctica introduce una sobrecarga computacional típicamente de dos a tres órdenes de magnitud sobre el entrenamiento individual. Cuando la distribución de los datos es estable y los hiperparámetros óptimos ya son conocidos de exploraciones anteriores, repetir la búsqueda en cada iteración del flujo es un coste evitable.

Esta observación no trivializa el valor de GridSearchCV ni de su variante aleatoria. La búsqueda exhaustiva es esencial la primera vez que se aborda un problema. Lo que este trabajo

sugiere es que, una vez encontrado un conjunto de hiperparámetros ganadores robustos, reutilizarlos en pipelines de producción, dashboards periódicos o sistemas de monitoreo continuo produce aceleraciones masivas sin sacrificar rendimiento.

4.2. Interpretabilidad de cada estrategia

No todas las estrategias evaluadas preservan la interpretabilidad en igual medida. El árbol fijo y el árbol con selección de variables por información mutua conservan el modelo original con reglas legibles sobre variables originales. La regresión logística con L1 y con GridSearch también mantiene interpretabilidad directa mediante sus coeficientes. La estrategia basada en PCA, en cambio, transforma las variables originales en combinaciones lineales que no tienen significado directo en el dominio: en el contexto educativo, un componente principal puede mezclar failures, absences y Medu en proporciones opacas, lo que dificulta justificar una predicción ante un consejo escolar.

En dominios sensibles esta distinción es importante. La estrategia 3 (selección por información mutua con árbol fijo) es probablemente la más equilibrada: reduce el espacio de variables a las diez más informativas, preserva los nombres originales, mantiene un árbol simple y pequeño, y entrena 16 veces más rápido que el baseline sin ninguna pérdida de F1 ni AUC. Para un sistema de alerta temprana en una institución educativa, esta combinación es difícil de mejorar.

4.3. Relación con el marco de Green AI

Schwartz et al. (2020) distinguen dos componentes del coste computacional: el costo de una única ejecución y el costo acumulado de un pipeline de investigación. En este trabajo se midió el primero. Pero si se considera el segundo, la ganancia es aún mayor: un proyecto que itera cien veces sobre el mismo problema ahorra 130 segundos (1.3 s x 100) al cambiar del baseline al árbol fijo. En flujos de CI/CD o en pipelines que reentrenan diariamente, la suma escala rápido.

Una consideración adicional que Schwartz et al. enfatizan es el reporte de la eficiencia como parte estándar de los resultados científicos. En este trabajo se han reportado tiempos absolutos en segundos medidos con reloj monotónico, lo que permite comparaciones directas con otros estudios sobre hardware similar. Esta práctica, que requiere poco esfuerzo adicional, es un

ingrediente simple y efectivo para contribuir a una cultura de Green AI en la investigación aplicada.

4.4. Limitaciones

El dataset es pequeño: 649 observaciones y 39 variables tras el one-hot encoding. Los tiempos absolutos son del orden de milisegundos a segundos, y el ruido de medición introduce variabilidad no despreciable en los experimentos 1 y 2. Las tendencias, sin embargo, son consistentes y se espera que se mantengan al escalar a datasets mayores, donde los factores relativos se amplifican.

Una limitación adicional es que todas las estrategias heredan los hiperparámetros ganadores de una búsqueda previa sobre el mismo dataset. Si se aplicara la estrategia 2 (árbol fijo) a un dataset nuevo sin ninguna búsqueda previa, los resultados no serían necesariamente óptimos. En la práctica, la recomendación es combinar una búsqueda exhaustiva inicial con ejecuciones posteriores simplificadas.

4.5. Conclusiones

Esta práctica refuerza tres ideas que son consistentes con el marco teórico de la Práctica 1 y con los hallazgos empíricos de la Práctica 2. Primera, el coste computacional del pipeline no reside en el algoritmo sino en los procedimientos accesorios como la búsqueda de hiperparámetros. Segunda, los modelos intrínsecamente interpretables no solo son competitivos en rendimiento, sino también extraordinariamente eficientes en coste, y esa eficiencia es fácil de preservar con estrategias simples como hiperparámetros fijos o selección de variables por información mutua. Tercera, la interpretabilidad y la eficiencia no son objetivos enfrentados: ambas se alinean de manera natural en los clasificadores pequeños y transparentes como los que Rudin (2019) defiende para dominios de alto riesgo.

5. Referencias bibliográficas

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.

Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. En A. Brito y J. Teixeira (Eds.), *Proceedings of 5th Future Business Technology Conference (FUBUTEC 2008)* (pp. 5-12). EUROSIS.

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3.ª ed.). Wiley. https://doi.org/10.1002/9781118548387

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence, 1*(5), 206-215. https://doi.org/10.1038/s42256-019-0048-x

Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2020). Green AI. *Communications of the ACM, 63*(12), 54-63. https://doi.org/10.1145/3381831

Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3645-3650. https://doi.org/10.18653/v1/P19-1355

Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological), 58*(1), 267-288.